Glosario básico: conceptos de IA del proyecto Ecommerce AI

Explicación sencilla de los términos técnicos de la Entrega 3. Cada concepto tiene una idea corta y, cuando ayuda, un ejemplo con el proyecto. No hace falta saber programar ni matemáticas para leerlo.

1. Ideas generales

Inteligencia artificial y aprendizaje automático. Programas que, en lugar de seguir reglas escritas a mano, encuentran patrones mirando muchos ejemplos. Aquí los ejemplos son 50,000 clientes.

Aprendizaje supervisado y no supervisado.

- *Supervisado:* el programa aprende con respuestas conocidas, como un alumno que estudia con un examen resuelto (por ejemplo, «este cliente se fue / este se quedó»).
- *No supervisado:* no hay respuestas; el programa propone un orden por su cuenta, como ordenar una caja de objetos sin que nadie diga las categorías. **Nuestro proyecto es no supervisado**: nadie le dice al modelo qué grupos existen.

Modelo. El resultado de «estudiar» los datos: una receta guardada que se puede aplicar a un cliente nuevo. Entrenar un modelo es hacerlo estudiar.

Entrenamiento y prueba (partición 80/20). Se reparten los clientes al azar: 80 % para que el modelo aprenda (entrenamiento) y 20 % que no ve mientras aprende (prueba). La prueba es el «examen sorpresa» para saber si de verdad funciona con clientes nuevos.

Sobreajuste (overfitting). Cuando el modelo memoriza los ejemplos en lugar de entender el patrón: saca 10 en lo que ya vio y falla con lo nuevo. Se detecta comparando el resultado en entrenamiento con el de prueba; si casi coinciden, no hay sobreajuste.

Hiperparámetro. Una perilla que se ajusta antes de entrenar (cuántos grupos, cuántas generaciones…). El modelo no la aprende sola; la elige el equipo.

Semilla. Un número que fija el «azar» del programa. Con la misma semilla se obtiene siempre el mismo resultado, lo que permite que otra persona repita el experimento y vea lo mismo (reproducibilidad).

Línea base (baseline). Un método simple contra el cual comparar. Si el modelo sofisticado no le gana a la línea base, no aporta. Usamos dos: las reglas RFM del dataset y repartir a los clientes al azar.

Inferencia. Usar el modelo ya entrenado con un cliente nuevo para obtener una respuesta. En el proyecto es la función `predecir()`.

2. Agrupar clientes

Agrupamiento (clustering). Repartir a los clientes en grupos de forma que los de un mismo grupo se parezcan entre sí y se diferencien de los otros.

K. El número de grupos que se pide. K = 3 significa «dame tres grupos».

Distancia. Qué tan diferentes son dos clientes según sus variables (días sin comprar, compras, satisfacción…). Dos clientes parecidos están «cerca»; dos muy distintos, «lejos».

Centroide. El «cliente promedio» o centro de un grupo, como el punto de reunión de todos sus miembros. Cada cliente nuevo se asigna al grupo cuyo centroide le queda más cerca. Los modelos de este proyecto son, en el fondo, K puntos centrales.

K-Means. El método clásico de agrupar: (1) pone K centros al azar; (2) cada cliente se une al centro más cercano; (3) cada centro se mueve al promedio de sus miembros; (4) repite hasta que ya no cambian. Es rápido y sencillo; lo usamos como modelo de comparación.

Algoritmo genético. Un método inspirado en la evolución. En vez de acomodar un solo juego de centros, mantiene una **población** de muchas propuestas y las va mejorando por **generaciones**:

- *Individuo (cromosoma):* una propuesta completa, aquí K centroides.
- *Aptitud:* una calificación de qué tan buena es la propuesta (usamos la silueta).
- *Selección por torneo:* se eligen tres propuestas al azar y se queda la mejor para «reproducirse».
- *Cruce:* se combinan dos propuestas buenas (cada centro viene de una u otra).
- *Mutación:* pequeños cambios al azar, para explorar opciones nuevas.
- *Elitismo:* las mejores propuestas pasan sin cambios a la siguiente generación, para no perderlas.

No garantiza el mejor resultado posible, pero explora muchas alternativas y permite exigir condiciones.

Tamaño mínimo de grupo. Una regla que le pusimos al algoritmo genético: ningún grupo puede quedar con menos del 12 % de los clientes (el compromiso es 10 %). Sin ella, el algoritmo hacía trampa: separaba unos pocos clientes extraños en un grupo diminuto y la calificación subía sin que sirviera de nada.

3. Preparar los datos

Variable (columna). Cada dato que se tiene de un cliente: edad, días desde su última compra, satisfacción, etc.

Estandarizar. Poner todas las variables en la misma escala (promedio 0, dispersión 1). Sin esto, una variable que se mide en miles (gasto) pesaría más que una que se mide del 1 al 5 (satisfacción) solo por sus unidades.

PCA (análisis de componentes principales). Una forma de resumir muchas variables en pocas «direcciones» que conservan casi toda la información, como una sombra que conserva la forma del objeto. Aquí se conserva el 95 % de la información.

Variable ordinal y variable de tipo «0/1». Para que un programa use texto hay que convertirlo en números. Si las opciones tienen orden (ingreso bajo < medio < alto) se les da un número en ese orden (*ordinal*). Si no tienen orden (color, país) se crea una columna por opción con 1 si aplica y 0 si no (*one-hot*).

Logaritmo. Un recurso matemático para «aplastar» valores muy grandes y raros, de modo que unos pocos extremos no dominen el resultado. Lo usamos en las razones por mes y por compra.

Valores atípicos. Datos muy distintos del resto (por ejemplo, 19 devoluciones con una sola compra). Pueden distorsionar las distancias.

Columna derivada y fuga de datos (data leakage). Una columna derivada se calcula a partir de otras (por ejemplo, `activity_status` sale de los días sin comprar). Usarla como entrada es como darle al alumno las respuestas del examen: el resultado se ve bien pero no aprendió nada. Por eso las 24 columnas derivadas **no** entran al modelo.

Imputación. Rellenar datos que faltan con un valor razonable (por ejemplo, «Ninguna» cuando un cliente no tiene red social).

RFM. Una regla clásica de marketing que califica a cada cliente por tres cosas: *Recency* (qué tan reciente compró), *Frequency* (cuántas veces compra) y *Monetary* (cuánto gasta). El dataset ya trae una clasificación RFM; la usamos como línea base.

4. Medir qué tan bueno es un agrupamiento

Silueta. Para cada cliente compara qué tan cerca está de su propio grupo contra qué tan cerca está del grupo ajeno más cercano. Va de −1 a 1: cerca de 1, bien ubicado; cerca de 0, en la frontera; negativo, probablemente en el grupo equivocado. La silueta del modelo es el promedio de todos los clientes. Una regla práctica: arriba de 0.5 los grupos son claros; entre 0.25 y 0.5, débiles; de 0.25 para abajo, casi no hay grupos reales.

Davies-Bouldin. Otra calificación de grupos: compara qué tan dispersos están los grupos contra qué tan separados quedan. **Menor es mejor.**

Calinski-Harabasz. Otra calificación más: relaciona la separación entre grupos con su compacidad. **Mayor es mejor.**

ARI (índice de Rand ajustado). Compara dos repartos del mismo conjunto de clientes. Vale 1 si son idénticos y 0 si se parecen tanto como dos repartos al azar. Lo usamos de dos formas: para ver si el modelo da el mismo reparto con semillas distintas (estabilidad) y para ver si recupera los grupos que escondimos en los datos generados.

Correlación (Pearson y Spearman). Qué tan relacionadas están dos variables: 0 significa que no tienen relación; 1 o −1, que una determina a la otra. *Pearson* mide relaciones en línea recta; *Spearman*, relaciones que siempre suben o siempre bajan. En este dataset casi todas las variables salieron en 0, lo que explica por qué no hay grupos claros.

Razón de correlación (η, «eta»). Lo mismo que la correlación, pero entre una **categoría** (como la frecuencia de compra) y un **número** (como los días sin comprar). Va de 0 a 1.

Ruido esperado. Aunque dos variables sean independientes, con datos de muestra la correlación nunca da exactamente 0; da un número pequeño por pura casualidad. Para 50,000 filas ese margen es de unas ±0.0045. Una correlación dentro de ese margen se considera cero.

5. Software y entrega

Serialización (joblib, `.joblib`). Guardar el modelo ya entrenado en un archivo para usarlo después sin volver a entrenar. Solo se deben abrir archivos de este tipo que uno mismo haya generado.

Pruebas automáticas. Programas cortos que comprueban que el código hace lo que debe (por ejemplo, que ninguna columna derivada entre al modelo). Se ejecutan con un solo comando.

Repositorio. La carpeta del proyecto con todo su historial de cambios, en GitHub. Quien lo clona puede repetir todo el trabajo.

Colab. Una plataforma de Google para ejecutar notebooks de Python en el navegador, sin instalar nada.

CSV y JSON. Formatos de archivo para datos: el CSV es una tabla (como Excel); el JSON es una estructura de datos con nombres. El modelo entrega sus resultados al dashboard en estos formatos.

`predecir()`. La función que recibe los datos de un cliente y devuelve a qué grupo pertenece, qué tan seguro está el modelo (`confianza`) y un estatus de alerta. Es el punto de unión entre el motor de IA y el dashboard.